

Le dispositif SNT, sous le capot

Ce que fait le serveur, comment les droits sont posés, et les gestes d'administration prêts à jouer. Sa version sans technique, destinée aux collègues, est le guide de prise en main.

1. L'architecture en un schéma

```
Pages du site  (GitHub Pages, HTML/CSS/JS sans build)
|
|  HTTPS, clé publique « anon »
v
Supabase  --  PostgreSQL managé, West EU (Paris)
|              - comptes et jetons de session
|              - API générée, filtrée ligne par ligne (RLS)
^
| se sert et repose son résultat
Worker de correction  (le PC de Loïc, tâche planifiée)
+-- API Mistral
```

Trois propriétés à garder en tête :

- **Le worker n'est pas dans la chaîne du site.** Il se réveille, prend les copies en attente, fait analyser, réécrit le résultat. Si le PC est éteint, le cours continue de fonctionner — les copies attendent.
- **La serrure est côté base, jamais côté page.** Le tableau de bord est une page publique sans secret : elle n'affiche rien sans compte professeur. C'est PostgreSQL qui refuse, pas le JavaScript qui masque.
- **Tout replie vers l'ouverture.** Sans compte, sans réseau, ou si la base ne répond pas, les séquences restent entièrement utilisables : rien n'est enregistré, rien n'est fermé. Le cours passe avant le dispositif.

2. Deux sortes de comptes

	Élève	Enseignant
Identifiant	Choisi par l'élève. Le client en fabrique une adresse interne <code>identifiant@snt.local</code> , jamais sollicitée.	Une vraie adresse, saisie telle quelle. Aucun domaine n'est ajouté.
Création	Par l'élève, avec le code de classe.	À la main : <i>Authentication</i> → <i>Users</i> → <i>Add user</i> , en cochant « Auto confirm user ».
Devient actif	Immédiatement — <code>rejoindre_classe()</code> crée sa fiche.	Quand <code>rattacher_enseignant()</code> l'a inscrit et rattaché.
Mot de passe	Haché par Supabase. On réinitialise , on ne lit jamais.	

LE SEUL DOCUMENT NOMINATIF

La base ne contient aucun nom. La correspondance identifiant → élève vit dans un fichier, sur le OneDrive de l'établissement, chargé à la demande dans l'onglet *Réglages* : lu sur l'appareil, jamais envoyé, effacé avec la session. **Sans ce fichier, un identifiant oublié est définitivement introuvable** — d'où la consigne de le faire noter sur la fiche bilan le jour de la création des comptes.

3. Ce que contient la base

Table	Ce qu'elle porte
classes	Le code à six caractères, le libellé, l'année, l'ouverture des inscriptions, le plafond d'avance.
eleves	Le pseudo et la classe. Unicité du pseudo <i>dans</i> une classe.
progression	L'état courant du parcours, en JSON.
evenements	L'historique, en ajout seul : on n'efface jamais.
reponses_libres	Les copies : texte, statut, analyse, commentaire.
reponses_versions	Chaque version remplacée, archivée automatiquement.
seances_faites	Le cahier de textes : séance, date réelle, notes, travail donné.
absences · jalons	Note de travail sur les absents ; dates cibles par thème.
enseignants · enseignants_classes	Qui est professeur, et qui enseigne où.
modeles_correction	La bibliothèque de phrases, cloisonnée par compte depuis l'origine.

Deux principes de conception qui expliquent le reste :

- **Le curseur n'est pas stocké**, il se déduit de `seances_faites`. Reculer d'une séance recolorise tout, sans script de rattrapage.
- **Les colonnes JSON se lisent toujours avec une valeur par défaut**. On ajoute des champs, on n'en renomme aucun, on n'en supprime aucun : le format évolue sans migration ni casse.

4. Le cloisonnement — qui voit quoi

Posé le 4 septembre 2026, quand l'équipe est passée d'un à cinq enseignants. Avant, un seul test existait — « ce compte est-il un professeur ? » — et il ouvrait tout le lycée.

Aujourd'hui, une table de rattachement dit qui enseigne où, et toutes les règles s'y adossent. Concrètement :

- Un enseignant ne lit que ses classes, ses élèves, leurs copies et leur progression.
- Il n'écrit le cahier de textes, les absences, les jalons et le plafond que sur ses groupes.
- **Une classe rattachée à personne est invisible de tous**. C'est ainsi que les deux classes du livret CFA sortent de ce tableau de bord : sans filtre, sans colonne, sans code — pendant que les apprentis continuent de s'inscrire et de travailler normalement.

LE POINT QUI N'ÉTAIT PAS ÉVIDENT

Fermer la lecture ne suffisait pas. Les corrections ne passent pas par une écriture directe mais par trois fonctions privilégiées — valider, renvoyer, rouvrir — qui **contournent les règles de lecture par construction**. Sans contrôle supplémentaire, un enseignant aurait pu valider n'importe quelle copie du lycée en connaissant son identifiant interne. Le cloisonnement rend cet identifiant introuvable, mais introuvable n'est pas interdit : les trois fonctions vérifient désormais la classe avant d'agir.

Pour vérifier à tout moment qu'aucune règle n'a fui, le fichier `bdd/schema/016-cloisonnement-enseignant.sql` se termine par sept contrôles. Le plus important liste toutes les règles des tables concernées et signale celles qui ne seraient pas cloisonnées.

5. Le parc de classes

Famille	Rôle	Plafond
Les 14 groupes de SNT	Un par groupe de la barrette, rattaché à son enseignant.	2 sur les trois groupes de Loïc, tout ouvert ailleurs.
Le groupe de démonstration commun	Rattaché aux cinq. C'est là que chacun crée ses élèves fictifs, sous le préfixe <code>zz-</code> .	Tout ouvert.
L'ancienne classe pilote	Rattachée à Loïc seul. Ses copies sont le seul matériel qui montre une vraie file de correction : les captures d'écran des guides s'y font.	Tout ouvert.
Les deux classes du livret CFA	Hors tableau de bord, par décision. Elles fonctionnent seules.	—

POURQUOI « TOUT OUVERT » PAR DÉFAUT

Le plafond se calcule à partir des séances déclarées faites. Un enseignant qui ne tient pas le cahier de textes a un journal vide : ses élèves seraient bloqués dès la deuxième séance et concluraient que le dispositif est cassé. Un plafond ne se pose donc que sur un groupe dont on clôture les séances.

6. Les gestes d'administration

Tous se jouent dans *SQL Editor* du tableau de bord Supabase. Les codes de classe réels ne figurent pas ici — la page est publique.

Ajouter un enseignant

Créer d'abord son compte dans *Authentication* → *Users*, « Auto confirm user » coché. Puis une seule ligne, qui retrouve le compte par son adresse, l'inscrit comme professeur et pose ses rattachements :

```
select public.rattacher_enseignant(  
  'adresse@exemple.fr',      -- son adresse de connexion  
  'NOM',                    -- ce qu'affiche le tableau de bord  
  array['CODE1','CODE2']);  -- ses groupes
```

Rejouable : relancée, elle met à jour le libellé et n'ajoute que les rattachements manquants. Si elle répond « Aucun compte pour... », c'est que le compte n'a pas été créé, ou que l'adresse diffère d'un caractère.

Retirer un groupe à un enseignant

```
delete from public.enseignants_classes ec  
using public.classes c, public.enseignants e  
where ec.classe_id = c.id and ec.auth_id = e.auth_id  
and c.code = 'CODE' and e.libelle = 'NOM';
```

Fermer ou rouvrir les inscriptions d'un groupe

À faire en général après la deuxième séance, pour éviter les inscriptions sauvages. Le code reste valable pour se reconnecter ; c'est la création de nouveaux comptes qui se ferme.

```
update public.classes set actif = false where code = 'CODE';  -- fermer  
update public.classes set actif = true  where code = 'CODE';  -- rouvrir
```

Faire le tour de l'année

```
-- qui enseigne où
select e.libelle, count(*) as groupes,
       string_agg(c.code, ' . ' order by c.code) as codes
from public.enseignants_classes ec
join public.enseignants e on e.auth_id = ec.auth_id
join public.classes      c on c.id      = ec.classe_id
group by e.libelle order by e.libelle;

-- combien d'élèves inscrits par groupe
select c.code, c.libelle, count(el.id) as inscrits
from public.classes c
left join public.eleves el on el.classe_id = c.id
group by c.code, c.libelle order by c.code;

-- les groupes que personne n'enseigne (le CFA doit y figurer)
select c.code, c.libelle from public.classes c
where not exists (select 1 from public.enseignants_classes ec
                  where ec.classe_id = c.id)
order by c.code;
```

Effacer les élèves fictifs en fin d'année

D'où la convention de préfixe. On les recense d'abord :

```
select el.pseudo, c.code
from public.eleves el join public.classes c on c.id = el.classe_id
where el.pseudo like 'zz-%' order by c.code, el.pseudo;
```

À FAIRE DANS L'INTERFACE, PAS EN SQL

Supprimer la ligne de `eleves` laisserait le compte d'authentification orphelin. C'est le compte qu'il faut supprimer, dans *Authentication* → *Users* : la fiche élève, sa progression et ses copies partent avec lui, en cascade. Le sens inverse ne marche pas.

7. La correction assistée

Le worker tourne sur le PC de Loïc, en tâche planifiée. À chaque réveil, il prend les vingt plus anciennes copies en attente **de toute la base**, sans distinction de classe, les fait analyser, et réécrit le résultat.

Il ne tranche rien. Le statut reste « en attente » : il attend une validation humaine. Ce qu'il produit est une proposition, affichée dans la file du professeur concerné.

POURQUOI IL N'A PAS BESOIN DE FILTRE

Il écrit globalement, mais le tri se fait à la **lecture** : une copie corrigée n'apparaît que dans la file de l'enseignant de l'élève, par le jeu des règles du §4. Aucune ligne de code de tri n'existe — et c'est précisément pour ça qu'elle ne peut pas se tromper.

Conséquence assumée : des copies d'élèves fictifs consomment des jetons comme les autres. À l'échelle de trois cent cinquante élèves, la dépense annuelle se compte en euros.

8. Entretien : sauvegardes et mise en pause

Quand	Quoi	À surveiller
Mercredi 18 h	Sauvegarde complète vers C:\Sauvegardes-SNT, et une ligne dans le journal.	Une ligne OK par semaine dans journal.log. Une ligne ECHEC, ou pas de ligne du tout, veut dire une sauvegarde muette.
Tous les jours, 12 h 30	Une requête, pour que le compteur d'inactivité reparte de zéro.	Une tâche manquée parce que le PC était éteint se rattrape au démarrage — mais un réveil en retard ne réveille rien.

SEPT JOURS

Sept jours consécutifs sans allumer le PC mettent le projet en pause. Rien n'est perdu et la relance se fait d'un clic, mais entre-temps le site ne répond plus : les élèves travaillent sans que rien ne soit enregistré. C'est le risque des vacances scolaires. À vérifier au retour, avant la reprise.

La sauvegarde couvre les tables et leurs données, **pas les comptes**, qui vivent dans une zone appartenant à Supabase. Une restauration dans un projet neuf retrouverait les fiches élèves mais plus les comptes qui les authentifient : à garder en tête pour toute migration.

9. Où vit quoi dans le dépôt

Chemin	Rôle
prof/index.html	Le tableau de bord.
assets/js/prof-api.js	Son client : connexion, lecture, écriture.
assets/js/progression.js	Le client des élèves. Partagé avec le livret CFA : le toucher pour la SNT touche le CFA.
bdd/schema/*.sql	Le schéma, numéroté, commenté, rejouable dans l'ordre.
supabase/migrations/	La même chose, horodatée, appliquée automatiquement au push.
ia-snt/	Le worker et ses grilles d'analyse.
prof/guide-*.html	Ces deux guides.

LE PIÈGE DU PUSH

Tout fichier .sql déposé dans supabase/migrations/ est **appliqué à la base au prochain push**, sans confirmation. Le bon ordre est donc toujours : exécuter à la main dans l'éditeur SQL, lire les vérifications de fin de fichier, *puis* déposer la copie horodatée et la déclarer déjà appliquée. L'inverse applique du SQL sans que personne n'en lise le résultat.

10. Ce qui reste ouvert

- **Les chapitres de physique-chimie** ne sont pas branchés : leur progression vit dans le navigateur de l'élève, et n'apparaît nulle part ici.
- **Les photos prises en classe** ne quittent pas le navigateur. Les remonter demanderait un espace de stockage, une table, des règles — et une durée de conservation décidée avant la première ligne de code.
- **Le bouton « voir mes versions »** côté élève n'existe pas encore, alors que l'archivage des versions fonctionne depuis le début.
- **La cible souveraine** reste un hébergeur français, à proposer à l'établissement. Le sol est déjà français ; c'est l'entreprise qui ne l'est pas.